

Université Bretagne Sud

Wireless Attack Network Testbed for Embedded Devices

Rapport

Projet WANTED

GHARBI Meysoun

Encadrant : Philippe Tanguy

2025/2026

Table des matières

1 Introduction générale **1**

2 Rappel du cahier des charges **1**
2.1 Objectifs du projet 1
2.2 Exigences fonctionnelles 1
2.3 Contraintes techniques et réglementaires 2

3 Démarche scientifique et technique **2**
3.1 Analyse du problème et modèle de menace 2
3.2 Choix techniques et justification 2
3.2.1 Faisabilité technique et matérielle 3
3.2.2 Simplicité de mise en œuvre et reproductibilité 3
3.2.3 Matériel accessible et pédagogiquement riche 3
3.2.4 Défi technique réaliste et formateur 3
3.3 Architecture du système 3
3.4 Mise en œuvre technique 4

4 Mise en œuvre du Banc de Test **4**
4.1 Configuration de la Passerelle (Gateway) 4
4.2 Configuration du Nœud Capteur (LoPy4) 4
4.3 Validation de la connectivité LoRaWAN 6

5 Configuration technique du nœud attaquant (Arduino) **6**
5.1 Préparation de l'environnement 6
5.2 Configuration des formats de clés (Endianness) 7

6 Réalisation des attaques **7**
6.1 Sniffing 7
6.2 Rejeu 8
6.3 Brouillage 8
6.4 Déni de Batterie 9
6.5 ACK Spoofing 11

7 Difficultés Rencontrées et Solutions **12**
7.1 Paramètres de Sécurité (Clés OTAA) 13
7.2 Synthèse de l'étape de mise en œuvre 13

8 Objectifs et indicateurs de réussite des attaques (Perspectives immédiates) **13**

9 Conclusion et bilan du projet **14**
9.1 Atteinte des objectifs 14
9.2 Retards et problèmes rencontrés 14
9.3 Analyse du diagramme de Gantt 14

10 Perspectives **14**

11 Réferences **15**

Table des figures

1 gateway connectée . . . 4
2 /dev/ttyACM0 . . . 5
3 DevEUI du loPy . . . 5
4 End device configuré dans ChirpSTack . . . 5
5 AppKey et DevEUI configurés par ChirpStack . . . 5
6 End device joined . . . 6
7 End device activé . . . 6
8 mapping . . . 7
9 Trames sniffées . . . 7
10 Trame analysée avec wireshark . . . 8
11 ABP activé . . . 8
12 FCNT désactivé . . . 8
13 Rejeu de trame en boucle . . . 8
14 Même MIC calculé que la trame rejouée . . . 8
15 Flux figé côté serveur . . . 9
16 JAM_SUCCESS . . . 9
17 Terminal LoRaWAN classeA . . . 10
18 structure d'un paquet . . . 10
19 Consommation d'énergie d'un terminal LoRaWAN en 2 modes [1] . . . 10
20 energy depletion attack . . . 11
21 ACK mode [2] . . . 11
22 Retransmission des paquets . . . 12
23 LoPy saturait la console . . . 12
24 diagramme de Gantt . . . 14

1 Introduction générale

Les réseaux LoRa/LoRaWAN sont aujourd'hui massivement déployés dans des domaines critiques tels que les smart cities, l'industrie, l'agriculture connectée ou encore la gestion d'infrastructures. Bien que LoRaWAN intègre des mécanismes de sécurité, de nombreuses vulnérabilités subsistent, notamment en raison de mauvaises configurations ou d'implémentations imparfaites.

Dans ce contexte, le projet **WANTED (Wireless Attack Network Testbed for Embedded Devices)** vise à fournir une plateforme expérimentale permettant d'orchestrer, d'exécuter et d'analyser des attaques sans fil sur des dispositifs embarqués, dans un environnement de laboratoire isolé.

Ce projet a pour but d'expérimenter différentes attaques sur un réseau LoRaWAN, notamment :

— **Sniffing** : interception des paquets LoRaWAN transmis sur le réseau.

— **Rejeu de paquets (replay attack)** : retransmission de paquets précédemment interceptés dans le but de tromper le réseau ou de provoquer des états incohérents.

— **Brouillage (jamming)** : perturbation volontaire du signal radio afin d'empêcher la communication entre les nœuds et le réseau.

— **Usurpation d'identité** : imitation d'un nœud légitime afin d'envoyer des données falsifiées ou malveillantes.

Au-delà de la reproduction d'attaques connues, ce travail vise également à analyser la littérature scientifique existante sur les attaques LoRa/LoRaWAN afin de tenter de reproduire ou d'adapter d'autres nouvelles attaques décrites dans des travaux de recherche récents. Cette approche permet d'inscrire le projet dans une démarche scientifique liant l'état de l'art aux expérimentations pratiques.

2 Rappel du cahier des charges

2.1 Objectifs du projet

L'objectif principal du projet est l'intégration d'attaques LoRa/LoRaWAN au sein de la plateforme **WANTED** afin de disposer d'un banc d'essai reproductible permettant :

— l'orchestration d'attaques sans fil,

— la génération de traces et de métriques exploitables,

— l'analyse de l'impact des attaques sur le réseau et les dispositifs.

2.2 Exigences fonctionnelles

Les exigences fonctionnelles définies pour le projet sont les suivantes :

— lancer, arrêter et superviser des attaques ,

— intégrer des attaques existantes dans WANTED,

— implémenter au moins deux attaques LoRa/LoRaWAN et essayer de les intégrer,

— générer des logs structurés au format JSON,

— fournir une documentation minimale.

2.3 Contraintes techniques et réglementaires

Le projet est soumis à plusieurs contraintes techniques et réglementaires :

— respect de la réglementation radio EU868,

— exécution des attaques dans un environnement isolé

3 Démarche scientifique et technique

3.1 Analyse du problème et modèle de menace

Une analyse préalable du système LoRa/LoRaWAN a permis d'identifier les actifs critiques à protéger, notamment le *network server*, le processus de *join* OTAA, l'intégrité des données, la passerelle et les terminaux. Le modèle de menace établi dans les livrables précédents se concentre exclusivement sur des attaques sans fil, en excluant volontairement les attaques physiques.

Les profils d'attaquants considérés incluent l'attaquant opportuniste, le concurrent malveillant et le saboteur ciblé. Les vecteurs d'attaque prioritaires incluent le sniffing, le rejeu de paquets, le brouillage radio (*jamming*), l'usurpation d'identité et l'épuisement de batterie.

Dans le cadre de ce projet, une première attaque issue de la littérature scientifique a été étudiée spécifiquement : **Attaque par Déni de Service Énergétique (Energy Depletion Attack) via l'exploitation du paramètre RX2 et de la fenêtre de réception**. Cette attaque, exploite le mécanisme de réception bidirectionnelle de LoRaWAN imposé par le protocole qui est l'ouverture de 2 fenêtres de réception après chaque uplink). L'attaquant envoie de fausses trame "downlink" avec un préambule valide (début du message LoRaWAN) adressées à une victime spécifique pendant l'ouverture de sa fenêtre de réception RW2. Le dispositif cible, pensant recevoir un message légitime du serveur (comme la vérification de la sécurité MIC se trouve à la toute fin de la trame), active son récepteur le recevoir et le traiter pour réaliser, trop tard, qu'il a consommé trop d'énergie pour un faux paquet.

Et une 2ème attaque :**ACK Spoofing (ou falsification d'accusé de réception)**. Contrairement à l'épuisement de batterie, cette attaque vise l'intégrité de la communication. L'attaquant utilise la même synchronisation sur la fenêtre RX2 pour injecter une courte trame de confirmation (ACK) factice. Le capteur, recevant ce signal, croit que sa donnée a bien été réceptionnée par le serveur et ne tente aucune retransmission, alors que le message original a pu être perdu ou intercepté. Cela crée un déni de service "invisible" où l'utilisateur perd des données sans que le système n'alerte sur une quelconque erreur.

3.2 Choix techniques et justification

Le choix de **LoRa/LoRaWAN** comme technologie cible s'explique par sa large adoption et par l'asymétrie importante entre le coût d'attaque et l'impact potentiel

La plateforme **ChirpStack** a été retenue comme *network server* en raison de son caractère open-source, de sa modularité et de sa large adoption dans la communauté académique.

Le choix des attaques issues de la littérature scientiques des attaques :

3.2.1 Faisabilité technique et matérielle

Cette attaque présente des **exigences matérielles minimales**, compatibles avec les contraintes d'un laboratoire pédagogique :

— Un serveur LoRaWAN local (`ChirpStack` sur un PC)

— Une passerelle LoRaWAN (Raspberry Pi 3 model B+ IC880A)

— Deux nœuds LoRa (un légitime (LoPy4), l'autre d'attaquant(ARDUINO))

3.2.2 Simplicité de mise en œuvre et reproductibilité

— **Version semi-simulée** : Un simple script Python peut inonder le serveur de requêtes de jointure forgées via le protocole UDP, sans même nécessiter d'émission radio. Cela permet de valider rapidement le principe de l'attaque.

— **Version "over-the-air" complète** : Nécessite uniquement de programmer un nœud attaquant pour émettre des trames `Join-Request` forgées, démontrant l'exploitation réelle de la vulnérabilité protocolaire.

Bien que légèrement plus exigeante que la précédente, cette attaque nécessite du matériel courant en laboratoire d'électronique :

3.2.3 Matériel accessible et pédagogiquement riche

— Une passerelle LoRaWAN

— Deux nœuds LoRa (légitime et attaquant)

— Un analyseur de consommation (multimètre ou oscilloscope, équipements standards)

Cette configuration permet d'introduire des **mesures quantitatives** (consommation énergétique) dans l'analyse de sécurité.

3.2.4 Défi technique réaliste et formateur

L'attaque présente un **défi de synchronisation temporelle** particulièrement instructif :

— **Complexité contrôlable** : Ce défi peut être résolu de plusieurs façons adaptées au contexte pédagogique :

  — Utilisation d'une **horloge commune** pour les deux nœuds (attaquant et victime)

  — Implémentation d'un **déclenchement par détection radio** de l'*uplink*

  — Synchronisation logicielle via le réseau local

3.3 Architecture du système

L'architecture globale du système repose sur les éléments suivants :

— *End-device* LoRa : un noeud légitime : LoPy4 et un noeud malveillant : Arduino dragino

— Passerelle LoRaWAN fournie par l'encadrant, incluant : Raspberry Pi 3 Model B + Module concentrateur LoRa IC880A

— Serveur réseau ChirpStack préconfiguré sur la passerelle.

3.4 Mise en œuvre technique

La mise en œuvre du projet s'est déroulée en plusieurs étapes :

— prise en main de l'architecture existante de WANTED,

— mise en place d'un environnement isolé incluant ChirpStack, une passerelle et des *end-devices* et sa configuration,

— exécution de scénarios d'attaque contrôlés (sniffing, brouillage simple).

Cette phase a présenté plusieurs défis, dont une méprise initiale sur la documentation à suivre, conduisant à explorer des configurations serveur non pertinentes (WalT, NFS) avant d'être réorienté par l'encadrant. Sur le bon chemin, des problèmes techniques sont survenus concernant la configuration de ChirpStack (identification de passerelle) et la synchronisation des équipements, problèmes qui ont été résolus par une étude documentaire approfondie et des tests itératifs.

4 Mise en œuvre du Banc de Test

Cette section détaille les étapes techniques pour établir la communication LoRaWAN entre le nœud capteur et le serveur réseau.

4.1 Configuration de la Passerelle (Gateway)

La passerelle est le cœur du réseau, faisant le pont entre la radio LoRa et l'IP.

— **Identification du matériel** : Nous avons utilisé un Raspberry Pi 3 couplé à un concentrateur IC880A.

— **Extraction de l'ID Gateway** : L'identifiant unique (EUI-64) est crucial pour ChirpStack. Nous l'avons généré à partir de l'adresse MAC de l'interface Ethernet `b8:27:eb:d5:b6:0e`.

— **Calcul de l'ID** : En insérant `FFFE` au centre de l'adresse MAC, nous avons obtenu le **Gateway ID** : `b827ebfffed5b60e`.

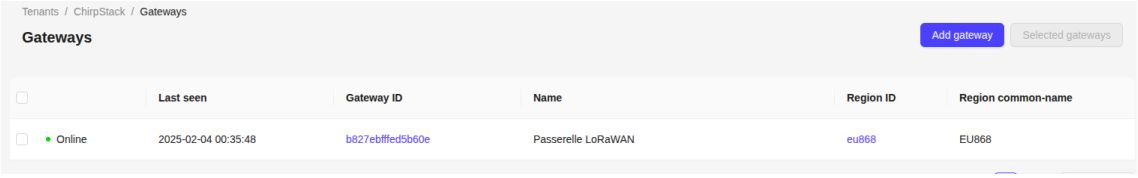


FIGURE 1 – gateway connectée

4.2 Configuration du Nœud Capteur (LoPy4)

Le LoPy4 agit comme le dispositif final (End-device) que nous allons ensuite attaquer.

— **Connexion au poste de travail** : Le module a été connecté via USB à un PC Lenovo. Il a été identifié sur le port série `/dev/ttyACM0` sous Linux.

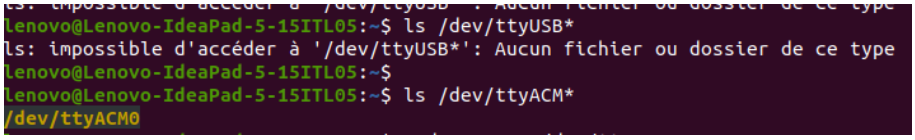


FIGURE 2 – /dev/ttyACM0

=> Afin de vérifier la bonne détection matérielle du nœud par le système d'exploitation Linux, une interrogation des périphériques série a été effectuée. Contrairement aux standards habituels (ttyUSB), le module a été identifié sur le port /dev/ttyACM0. Cette étape de diagnostic a été cruciale pour configurer l'environnement de développement VS Code et l'extension Pymakr.

— **Extraction du DevEUI** : Pour enregistrer le nœud dans ChirpStack, nous avons dû extraire son identifiant matériel unique via un script MicroPython utilisant `lora.mac()`.

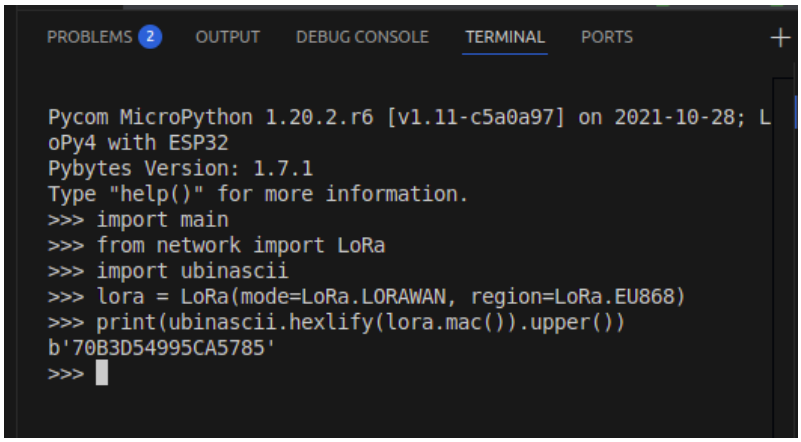


FIGURE 3 – DevEUI du loPy

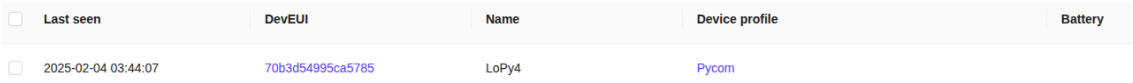


FIGURE 4 – End device configuré dans ChirpSTack

Une fois le DevEUI récupéré, nous avons implémenté le script de connexion utilisant la méthode d'activation OTAA. L'élément central de ce script est l'AppKey, générée sur le serveur ChirpStack et reportée dans le code pour permettre le chiffrement des échanges

```
# create an OTAA authentication parameters, change them to the provided creden
app_eui = ubinascii.unhexlify('0000000000000000') #app key non trouvée
app_key = ubinascii.unhexlify('bae64ce2ce34fd94c24f8ec55c233ecb')
#uncomment to use LoRaWAN application provided dev_eui
dev_eui = ubinascii.unhexlify('70B3D54995CA5785')
```

FIGURE 5 – AppKey et DevEUI configurés par ChirpStack

4.3 Validation de la connectivité LoRaWAN

Apès les changements des AppKey et DevEUI dans le code, nous avons supprimé le contenu du mémoire flash du LoPy et flashé le nouveau code.



FIGURE 6 – End device joined

Une fois l'infrastructure est configuré, nous voyons un JoinRequest (côté passerelle) et un JoinAccept (côté noeud) ainsi qu'une trame monantante (uplink) et une trame descendante (downlink).

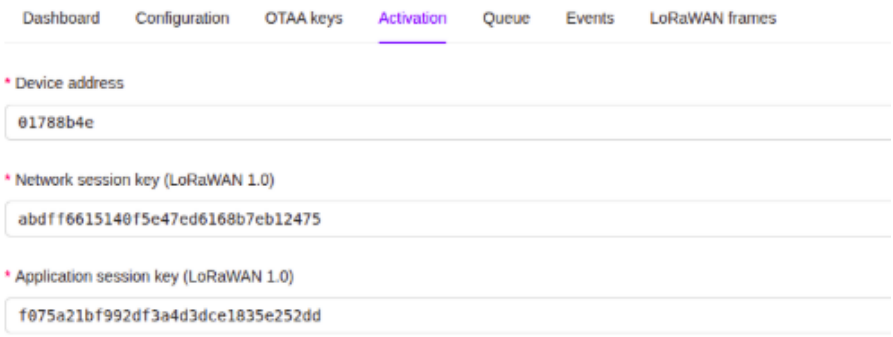


FIGURE 7 – End device activé

Dans l'onglet activation de ChirpStack, nous voyons que le end device est activé et il a maintenant une adresse dynamique (Device address) et des clés de session (Network session key et Application session key), ce qui prouve que l'activation OTAA est terminée.
Et dans l'onglet LoRaWAN frames du device, nous voyons des messages UnconfirmedDataUp. Ce sont les données du capteur (Humidité) arrivent sur le serveur.

5 Configuration technique du nœud attaquant (Arduino)

Pour réaliser les attaques, nous avons utilisé un Arduino équipé d'un module radio LoRa, piloté par la bibliothèque MCCI LoRaWAN LMIC. Cette bibliothèque est critique car elle permet un contrôle bas niveau de la radio.

5.1 Préparation de l'environnement

— Configuration du fichier `lmic_project_config.h` pour forcer la fréquence sur EU868, correspondant à la zone géographique de notre passerelle Raspberry Pi.

— La communication entre l'Arduino et la puce radio (RFM95) avec un câblage spécifique (NSS sur pin 10, RST sur 9, et DIO sur 2, 6, 7) est déjà fournie

```
// Pin mapping
const lmic_pinmap lmic_pins = {
    .nss = 10,
    .rxtx = LMIC_UNUSED_PIN,
    .rst = 9,
    .dio = {2, 6, 7},
};
```

FIGURE 8 – mapping

5.2 Configuration des formats de clés (Endianness)

Une difficulté technique majeure en LoRaWAN est l'ordre des octets. Pour que l'Arduino puisse "dialoguer" avec le serveur ChirpStack, il faut convertir les clés :

— **Little Endian (LSB)** pour le `DevEUI` et le `JoinEUI`.

— **Big Endian (MSB)** pour l'`AppKey`.

6 Réalisation des attaques

Une fois le réseau est validé, nous passons à la phase offensive :

6.1 Sniffing

L'attaque par **Sniffing** est la forme la plus élémentaire d'attaque passive. Elle consiste à écouter le support radio pour capturer les trames échangées entre le terminal (End-Device) et la passerelle (Gateway) sans en perturber le flux.

Dans le cadre de ce projet, nous avons utilisé le nœud attaquant (Arduino + Dragino) configuré en mode "promiscuous" pour intercepter les signaux LoRa circulant sur la fréquence 868.3 MHz.

```
Paquet re◆çu !
Taille du paquet : 27
RSSI : -75
Donn◆ées re◆çues (hex) : 40 94 67 3D 01 00 84 06 02 AD F7 4A 76 BF DE 8A 8F 53

Paquet re◆çu !
Taille du paquet : 27
RSSI : -75
Donn◆ées re◆çues (hex) : 40 94 67 3D 01 00 8B 06 02 9A D8 16 25 C1 45 42 35 83
```

FIGURE 9 – Trames sniffées

Nous analysons les trames capturées avec **Wireshark** pour identifier des vulnérabilités ou les utiliser pour des attaques ultérieures.

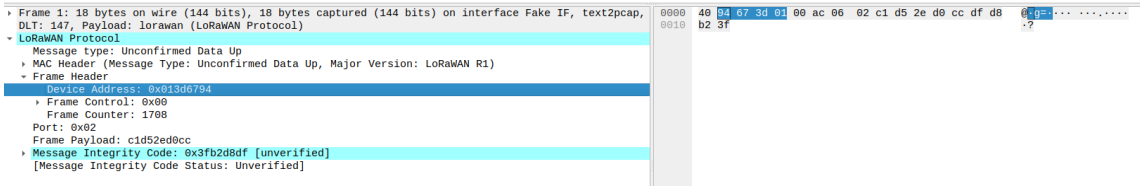


FIGURE 10 – Trame analysée avec wireshark

6.2 Rejeu

Maintenant que nous sommes en possession d'une trame valide, nous passons au rejeu.

Les attaques par rejeu démontrent que la sécurité de LoRaWAN ne repose pas uniquement sur le protocole lui-même mais sur la rigueur de sa configuration.
Pour réaliser cette attaque, nous avons utilisé le mode **ABP** qui est moins sécurisé et nous avons désactivé **la validation de frame counter**



FIGURE 11 – ABP activé

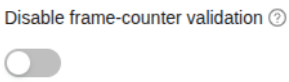


FIGURE 12 – FCNT désactivé

Après, nous avons rejoué une trame en boucle

```
Rejeu de la trame : 40 31 CC 9E 01 00 14 02 02 78 B7 11 B3 A6 55 E6 4B
: 40 31 CC 9E 01 00 14 02 02 78 B7 11 B3 A6 55 E6 4B
Rejeu de la trame : 40 31 CC 9E 01 00 14 02 02 78 B7 11 B3 A6 55 E6 4B
LoRa Replayer
Pr◆êt ◆à rejouer le paquet en boucle...
Rejeu de la trame : 40 31 CC 9E 01 00 14 02 02 78 B7 11 B3 A6 55 E6 4B
Rejeu de la trame : 40 31 CC 9E 01 00 14 02 02 78 B7 11 B3 A6 55 E6 4B
```

FIGURE 13 – Rejeu de trame en boucle

Nous remarquons, dans les logs de chirpstack, que la même vérification de sécurité (MIC) que celle de la trame rejouée, cela veut dire que le serveur a accepté la trame rejouée.

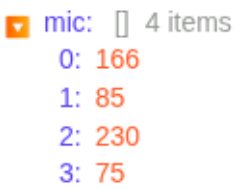


FIGURE 14 – Même MIC calculé que la trame rejouée

=> Cette attaque met en évidence que des vulnérabilités critiques peuvent être introduites par des négligences humaines lors de la configuration du réseau ou des terminaux.

6.3 Brouillage

Cependant, si le serveur est bien protégé, l'attaquant peut passer au brouillage **Jamming** (niveau physique).

Cette attaque cherche à rendre le réseau totalement indisponible. À partir d'un noeud attaquant qui émet en boucle à la **puissance maximale** sur la **fréquence du réseau** ce qui sature l'antenne du passerelle :

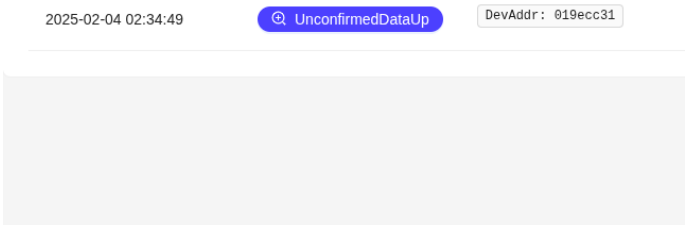


FIGURE 15 – Flux figé côté serveur

Nous voyons **côté serveur** que le flux est figé (il n'y a plus des lignes des uplinks qui s'affichent) car la paserelle est incapable de distinguer les vraies trames au milieu du bruit. **Côté attaquant**, le terminal confirme l'envoi continue avec le message "JAM_SUCCESS"

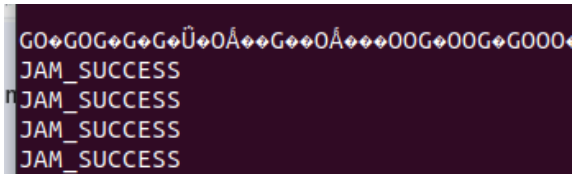


FIGURE 16 – JAM_SUCCESS

Ces attaques se concentrent sur la couche applicative et la sécurité des données. Cependant, la littérature récente s'interesse à l'exploitation de la ressource la plus critique des **LPWAN**, qui est l'énergie. Nous allons voir comment transformer une vulnérabilité protocolaire en un véritable **Déni de Batterie** (DOB)

6.4 Déni de Batterie

Pour comprendre cette attaque, il faut savoir que chaque terminal LoRaWAN est obligé d'ouvrir 2 fenêtres de réception- RW1 et RW2- après chaque envoi de données (uplinks) (comme montre la figure Figure 16 : Terminal LoRaWAN), c'est une régle du protocole, même s'il y a rien à recevoir. Normalement, si le terminal ne détecte rien, il referme sa fenêtre très vite pour économiser sa batterie, mais nous, nous exploitons une faille : le début d'un message LoRa (préambule) n'est pas crypté. Donc, l'attaquant envoie des faux messages avec un préambule valide pile au moment où le terminal ouvre sa 2ème fenêtre. Comme la vérification de sécurité (MIC) ne se trouve qu'à la toute fin du message (comme montre la figure Figure 17- Structure d'un paquet), le terminal est obligé de rester allumer et de consommer de l'énergie pour lire tout le paquet avant de réaliser, trop tard, qu'il était faux.

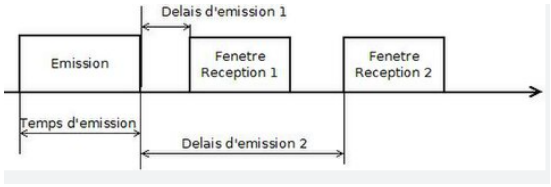


FIGURE 17 – Terminal LoRaWAN classeA

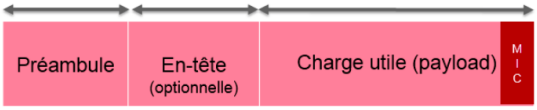


FIGURE 18 – structure d'un paquet

La courbe de la figure, Figure 18 : Consommation d'énergie d'un terminal LoRaWAN en 2 modes, présente le profil de consommation d'un terminal LoRaWAN en mode de fonctionnement normal comparé à un terminal subissant une attaque de type "Energy Depletion"

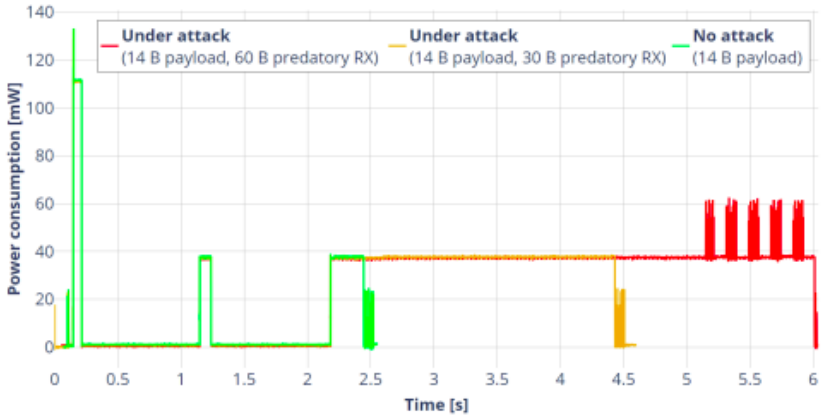


FIGURE 19 – Consommation d'énergie d'un terminal LoRaWAN en 2 modes [1]

En mode de fonctionnement normal (No attack, courbe verte), le terminal LoRaWAN Classe A optimise sa consommation en restant la majeure partie du temps en mode veille profonde. Lors d'un cycle de transmission, on observe un pic de consommation élevé correspondant à l'envoi du paquet (TX), suivi de deux fenêtres de réception courtes (RX1 et RX2) avant que le module ne retombe immédiatement à un niveau d'énergie quasi nul pour préserver la batterie. Cependant, lors d'une attaque de type "Energy Depletion" (Under attack, courbes jaune et rouge), le comportement change radicalement. Sous l'effet d'une charge utile malveillante ou "prédatrice" lors de la réception, le terminal est forcé de rester actif beaucoup plus longtemps pour traiter des données inutiles ou effectuer des retransmissions fréquentes. Cette activité prolongée empêche le retour en mode veille, provoquant un plateau de consommation élevé qui épuise prématurément les ressources énergétiques du capteur.

Comme nous disposons pas d'un analyseur de puissance pour mesurer la consommation. Nous avons fait un petit calcule pour synchroniser l'attaquant Arduino pour qu'il injecte un faux paquet EXACTEMENT à l'ouverture de la fenêtre RW2 du lopy4. La durée de cette fenêtre devient alors l'indicateur de l'épuisement énérgetique provoqué.



FIGURE 20 – energy depletion attack

Les logs confirment que la radio est forcée de rester en état de haute consommation pendant plus de 3000 ms, contre une fraction de seconde en temps normal. Ce maintien artificiel de l'activité radio crée un plateau de consommation d'environ 40 mW qui s'étend sur plusieurs secondes, multipliant ainsi drastiquement l'énergie dissipée pour chaque message envoyé et réduisant irrémédiablement l'autonomie du capteur.

6.5 ACK Spoofing

L'attaquant peut aller plus loin que cela, l'idée n'est plus seulement de détruire l'objet physiquement mais de prendre contrôle de ce qu'il croit être vrai. Ici, intervient l'attaque de ACK Spoofing.

Dans LoRaWAN, un end device peut demander un ACK pour ses transmissions uplinks. En absence de cet ACK, le device retransmet le message plusieurs fois (comme montre la figure ci dessous), ce qui augmente fortement sa consommation énérgetique. Un attaquant peut exploiter ce mécanisme en bloquant les ACK légitimes, puis en injectant un faux ACK afin de convaincre le device que son message a été reçu provoquant ainsi une perte silencieuse de données et un épuisement de la batterie.

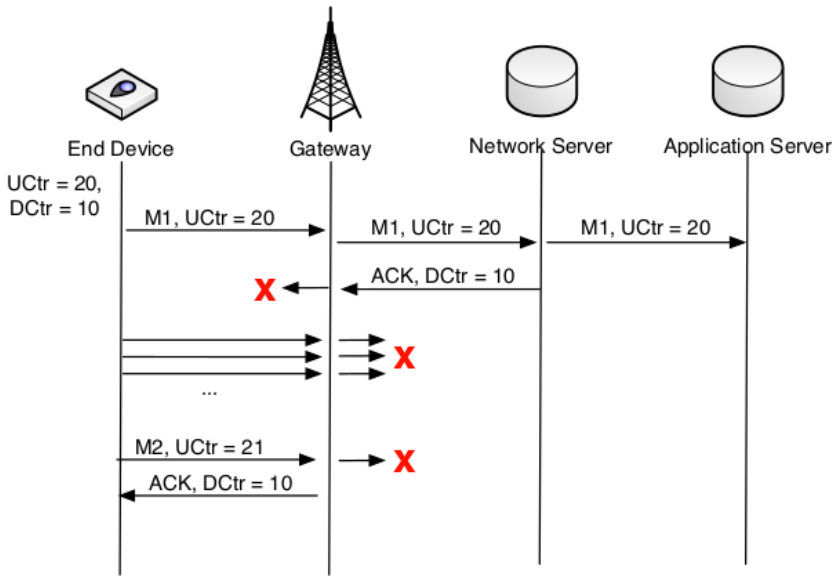


FIGURE 21 – ACK mode [2]

L'injection d'un message ACK est rendue possible par le fait que dans LoRaWAN, un ACK n'est pas cryptographiquement lié à un message uplink précis donc un ACK injecté au bon moment peut être accepté par le device.

Le lopy4 a été configuré pour demander un ACK après chaque uplink afin d'augmenter la consommation énérgetique, vue que le terminal LoRaWAN consomme plus d'énérgie lors de TX. Le noeud attaquant, de sa part, bloque les ACK légitimes et injecte son ACK.

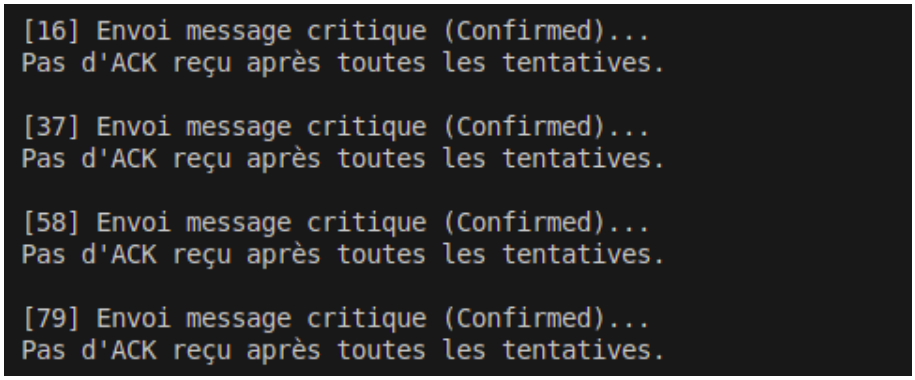


FIGURE 22 – Retransmission des paquets

Il y avait, toutefois, des difficultés au niveau de l'injection ACK en raison de **Frequency Hopping** nous sommes pas arrivés à synchroniser la fréquence de transmission. Mais, nous avons fait l'épuisement d'énérgie. Comme indique la Figure 21, il y a 21s mises pour chaque retransmission ce qui consomme trop l'énérgie.

7 Difficultés Rencontrées et Solutions

— **Visibilité du périphérique USB** : Le port série n'était pas initialement visible avec `ls /dev/ttyUSB*`. La solution a été d'identifier que le module utilisait le pilote ACM (`/dev/ttyACM0`) et de configurer l'extension Pymakr dans VS Code.

— **Accès à la console (REPL)** : Lors de la phase de test, nous avons identifié que le module LoPy4 conservait en mémoire flash une ancienne version du script main.py. Cette persistance empêchait la prise en compte des nouveaux identifiants (AppKey et DevEUI), causant un échec systématique de l'authentification sur le serveur ChirpStack en saturant la console avec des messages "Not yet joined", empêchant la saisie de commandes ainsi qu'un échec de la procédure Join(OTAA).



FIGURE 23 – LoPy saturait la console

Pour résoudre ce conflit, nous avons formaté la partition flash en utilisant la commande `os.fsformat('/flash')` , flashé le nouveau code qui contient les bonnes clés

et redémarré le matériel

— **Échec de la procédure Join (OTAA)** : Bien que les trames `"Join Request"` soient visibles dans ChirpStack, l'activation restait bloquée. Cela a permis de souligner l'importance de la concordance parfaite entre le `DevEUI`, le `JoinEUI` (`AppEUI`) et l'`AppKey`.

— **Anomalie d'horodatage (Infrastructure)** : Une fois la connexion établie (statut `"Joined"`), nous avons constaté que les trames reçues sur ChirpStack affichaient une date erronée (février 2025). Le diagnostic a permis d'isoler ce problème au niveau de la passerelle LoRaWAN : celle-ci n'est pas synchronisée via le protocole NTP (*Network Time Protocol*). Bien que ce décalage temporel n'entrave pas la transmission des données en temps réel, il souligne un défaut de maintenance de l'infrastructure réseau utilisée.

7.1 Paramètres de Sécurité (Clés OTAA)

Pour finaliser la connexion, les paramètres suivants ont été configurés :

— **JoinEUI** : Fixé à `0000000000000000` par convention pour ce labo.

— **AppKey** : Clé de 128 bits générée aléatoirement sur ChirpStack et copiée dans le firmware du LoPy4 pour le chiffrement AES.

7.2 Synthèse de l'étape de mise en œuvre

La mise en œuvre de la communication entre le nœud légitime (LoPy4) et la passerelle (Raspberry Pi 3) a constitué la phase la plus délicate et la plus chronophage de ce projet. Cette étape a nécessité une compréhension profonde des interactions entre les couches matérielles et les protocoles réseau (OTAA, gestion des clés AES-128).
Toutefois, ce temps investi a été déterminant : une fois la liaison établie et sécurisée, la reproduction des attaques est devenue une procédure beaucoup plus directe. L'essentiel de ce travail réside dans la maîtrise de l'infrastructure LoRaWAN ; comprendre comment les trames sont générées et validées par le serveur ChirpStack permet désormais d'identifier précisément les vecteurs de vulnérabilité.

8 Objectifs et indicateurs de réussite des attaques (Perspectives immédiates)

L'implémentation des scénarios d'attaque sur ce banc de test a pour but de démontrer la faisabilité de l'orchestration d'attaques LoRaWAN via le framework WANTED.
Les outils de capture et les scripts Arduino configurés précédemment permettent de générer des logs détaillés. Nous avons pu extraire des métriques exploitables pour quantifier l'efficacité de l'attaque, notamment :

— Le nombre de paquets capturés lors des phases de **sniffing**.

— Le taux de paquets perdus (**Packet Loss**) côté passerelle lors des phases de **brouillage**.

— Le succès des tentatives de **rejeu de trames**.

— L'épuisement de batterie lors **Déni de batterie** et **ACK spoofing**

Les résultats attendus ont confirmé les hypothèses formulées lors de l'étude théorique

9 Conclusion et bilan du projet

9.1 Atteinte des objectifs

Au regard des objectifs définis dans le cahier des charges, le projet a quasi permis d'atteindre une grande partie des attendus. La reproduction d'attaques LoRa/LoRaWAN existantes ainsi que la recherche des nouvelles attaques pour les intégrer dans WANTED ont été réalisées avec succès.

9.2 Retards et problèmes rencontrés

Des retards ont été observés, principalement dus à la complexité de l'environnement radio. Ces difficultés ont néanmoins permis d'approfondir la compréhension des contraintes réelles liées aux attaques sans fil.

9.3 Analyse du diagramme de Gantt

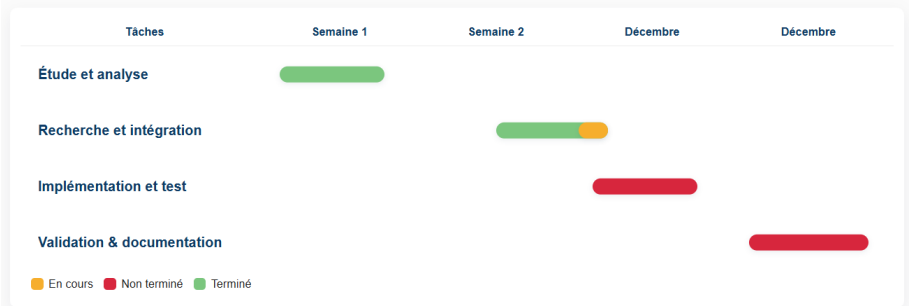


FIGURE 24 – diagramme de Gantt

L'analyse du diagramme de Gantt met en évidence des écarts entre les temps estimés et le temps réellement consacré à certaines tâches, notamment lors des phases d'intégration et de validation. Ces écarts s'expliquent par des itérations supplémentaires nécessaires pour stabiliser l'environnement expérimental.

10 Perspectives

Le travail réalisé a permis d'établir une base solide pour l'étude de la sécurité LoRaWAN. À court terme, les perspectives du projet incluent l'implémentation d'attaques supplémentaires et l'automatisation des scénarios. À long terme, l'intégration de mécanismes de détection et de contre-mesures directement au sein de la plateforme WANTED.

11 Réferences

[1] Mikhaylov, K., Voznak, M., Fujdiak, R., Malina, L., Pouttu, A., & Mlynek, P. (2019). Energy Attack in LoRaWAN : Experimental Validation.

[2] X. Yang, E. Karampatzakis, C. Doerr, and F. Kuipers, "Security Vulnerabilities in LoRaWAN," Delft University of Technology, Delft, The Netherlands, Tech. Rep.